981. Time Based Key-Value Store Medium

Acceptance: 49.7% | LeetCode

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the `TimeMap` class:

- `TimeMap()` Initializes the object of the data structure.
- `void set(String key, String value, int timestamp)` Stores the key `key` with the value `value` at the given time `timestamp`.
- `String get(String key, int timestamp)` Returns a value such that `set` was called previously, with `timestamp_prev <= timestamp`. If there are multiple such values, it returns the value associated with the largest `timestamp_prev`. If there are no values, it returns `""`.

Example 1:

```
**Input**
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3],
["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
**Output**
[null, null, "bar", "bar", null, "bar2", "bar2"]

**Explanation**
TimeMap timeMap = new TimeMap();
timeMap.set("foo", "bar", 1);  // store the key "foo"
and value "bar" along with timestamp = 1.
timeMap.get("foo", 1);         // return "bar"
timeMap.get("foo", 3);         // return "bar", since
there is no value corresponding to foo at timestamp 3
and timestamp 2, then the only value is at timestamp 1
is "bar".
timeMap.set("foo", "bar2", 4); // store the key "foo"
and value "bar2" along with timestamp = 4.
timeMap.get("foo", 4);         // return "bar2"
timeMap.get("foo", 5);         // return "bar2"
```

Constraints:

- `1 <= key.length, value.length <= 100`
- `key` and `value` consist of lowercase English letters and digits.
- `1 <= timestamp <= 107`
- All the timestamps `timestamp` of `set` are strictly increasing.
- At most `2 * 105` calls will be made to `set` and `get`.

Claude Opus Solution

Time Based Key-Value Store — Solution Guide

1. Intuition

The key observation comes from a crucial constraint:

> **All the timestamps `timestamp` of `set` are strictly increasing.**

This means that for any given key, the timestamps we store are **already sorted** in ascending order (since we append them in the order they arrive).

When we want to find the value at a query timestamp `t`, we need the **largest stored timestamp that is ≤ t**. This is a classic **binary search** pattern: searching for a "floor" value (the rightmost element satisfying a condition) in a sorted array.

The pattern: Hash Table (for O(1) key lookup) + Binary Search (for O(log n) timestamp lookup).

2. Approach

```
SET(key, value, timestamp):
    Append [timestamp, value] to map[key]
    (Array stays sorted because timestamps are strictly increasing)

GET(key, timestamp):
    entries = map[key]
    Binary search for the rightmost entry where entry.timestamp <= timestamp

    Initialize answer = ""
    left = 0, right = len(entries) - 1
    while left <= right:
        mid = (left + right) // 2
        if entries[mid].timestamp <= timestamp:
            answer = entries[mid].value   # candidate found, try to find a later one
            left = mid + 1                # search right half for a larger valid ts
        else:
```

```
            right = mid - 1              # ts too big, search left half
    return answer
```

The binary search finds the **floor**: we keep moving right whenever we find a valid timestamp (to maximize the timestamp), and move left whenever the current timestamp is too large.

---

3. Python Solution

```python
import collections
from bisect import bisect_right


class TimeMap:
    def __init__(self):
        # Map each key to a list of (timestamp, value) pairs.
        # Because set() is always called with strictly increasing timestamps,
        # this list is automatically kept sorted by timestamp.
        self.store = collections.defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        # Simply append the new (timestamp, value) pair.
        # O(1) amortized; sorted order is preserved by the problem's guarantee.
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        # Retrieve the sorted list of (timestamp, value) for this key.
        entries = self.store[key]

        answer = ""              # default: no valid value found
        left, right = 0, len(entries) - 1

        # Binary search for the RIGHTMOST entry whose timestamp <= query timestamp.
        while left <= right:
            mid = left + (right - left) // 2   # avoid potential overflow (good habit)

            if entries[mid][0] <= timestamp:
                # This entry is valid (its timestamp does not exceed the query).
                # Record it as a candidate, then look right for an even larger
                # valid timestamp (we want the LARGEST timestamp_prev <= timestamp).
                answer = entries[mid][1]
                left = mid + 1
            else:
                # Timestamp is too large; discard right half and search left.
                right = mid - 1

        return answer
```

Cleaner alternative using `bisect`

```python
import collections
from bisect import bisect_right


class TimeMap:
    def __init__(self):
        # Keep timestamps and values in parallel sorted lists per key.
        self.times = collections.defaultdict(list)   # sorted timestamps
        self.values = collections.defaultdict(list)  # corresponding values

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.times[key].append(timestamp)
        self.values[key].append(value)

    def get(self, key: str, timestamp: int) -> str:
        ts = self.times[key]
        # bisect_right returns the insertion point to the RIGHT of any equal element.
        # So idx - 1 is the index of the largest timestamp <= our query timestamp.
        idx = bisect_right(ts, timestamp)
        if idx == 0:
            return ""             # all stored timestamps are > query timestamp
        return self.values[key][idx - 1]
```

4. Complexity Analysis

Let `n` be the number of entries stored for a particular key.

Operation	Time	Space
`__init__`	O(1)	O(1)
`set`	O(1) amortized	O(1) extra per call
`get`	**O(log n)**	O(1)

- **`set` — O(1):** We just append to a list. The sorted order is free thanks to the strictly-increasing-timestamp guarantee. (Without this guarantee, we'd need an insertion that costs O(n) or a balanced BST.)
- **`get` — O(log n):** Binary search over the sorted entries.
- **Overall Space — O(N):** where `N` is the total number of `set` calls across all keys; we store one pair per call.

5. Alternative Approaches

1. **Linear scan (naive):** In `get`, iterate the list backwards and return the first entry with `timestamp ≤ query`. This is **O(n)** per `get` and will likely TLE given up to 2×10^5 calls. The binary search optimization is the intended improvement.
2. **`bisect` module (shown above):** Using Python's built-in `bisect_right` on a separate sorted timestamps list is cleaner and less error-prone than hand-written binary search. Same O(log n) complexity, but offloads the tricky boundary logic to a battle-tested library function. **This is the recommended production approach.**
3. **Sorted dictionary / balanced BST (`sortedcontainers.SortedDict`):** If timestamps were **not** guaranteed to be increasing, you'd need a structure that maintains sorted order on insertion. `SortedDict` supports `bisect`-style operations and would give O(log n) for both `set` and `get`. Overkill here, but the right tool when the monotonicity guarantee is absent.

Key Takeaways

- **Recognize sorted-by-construction data** → enables binary search without explicit sorting.
- **"Find largest element ≤ target"** is the *floor* / *rightmost-valid* binary search template — memorize it.
- Prefer `bisect_right(arr, x) - 1` for finding the floor element; it's concise and bug-resistant.

Codex Solution

1. Intuition

For each `key`, we may store many `(timestamp, value)` pairs.

The important observation is:

> `set` timestamps are strictly increasing, so for each key, its timestamps are stored in sorted order if we simply append them.

Therefore, for a query:

```
get(key, timestamp)
```

we need to find the largest stored timestamp `<= timestamp`.

This is a classic **binary search for the rightmost valid value**.

So the data structure is:

```
key -> sorted list of (timestamp, value)
```

Example:

```
"foo" -> [(1, "bar"), (4, "bar2")]
```

For:

```
get("foo", 3)
```

we binary search for the largest timestamp `<= 3`, which is `1`, so return `"bar"`.

2. Approach

Data Structure

Use a hash map:

```
store[key] = list of (timestamp, value)
```

Because timestamps are increasing, each list remains sorted by timestamp.

`set(key, value, timestamp)`

Append the new pair to the list for that key.

Pseudocode

```
function set(key, value, timestamp):
    if key does not exist:
        create empty list for key

    append (timestamp, value) to store[key]
```

`get(key, timestamp)`

Binary search over the list for `key`.

We want the rightmost timestamp `<= timestamp`.

Pseudocode

```
function get(key, timestamp):
    if key does not exist:
        return ""

    values = store[key]

    left = 0
    right = len(values) - 1
    answer = ""

    while left <= right:
        mid = (left + right) // 2

        if values[mid].timestamp <= timestamp:
            answer = values[mid].value
            left = mid + 1
        else:
            right = mid - 1

    return answer
```

---

3. Python Solution

```python
from collections import defaultdict
from typing import List, Tuple


class TimeMap:
    def __init__(self):
        """
        Initialize the time-based key-value store.

        self.store maps:
            key -> list of (timestamp, value)

        Example:
            {
```

```
                "foo": [(1, "bar"), (4, "bar2")]
            }

        The list is sorted by timestamp because set() calls have
        strictly increasing timestamps.
        """
        self.store = defaultdict(list)

    def set(self, key: str, value: str, timestamp: int) -> None:
        """
        Store the given value for the key at the given timestamp.

        Since timestamps are strictly increasing, we can simply append.
        This keeps the list sorted by timestamp.
        """
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        """
        Return the value associated with the largest timestamp_prev
        such that timestamp_prev <= timestamp.

        If no such timestamp exists, return "".
        """

        # If the key was never inserted, there is no valid value.
        if key not in self.store:
            return ""

        values = self.store[key]

        # Binary search for the rightmost timestamp <= given timestamp.
        left = 0
        right = len(values) - 1

        # Stores the best valid answer found so far.
        answer = ""

        while left <= right:
            mid = left + (right - left) // 2
            mid_timestamp, mid_value = values[mid]

            if mid_timestamp <= timestamp:
                # This value is valid.
                # But there may be a later valid timestamp to the right.
                answer = mid_value
                left = mid + 1
            else:
                # Timestamp is too large, search left side.
                right = mid - 1

        return answer
```

---

4. Complexity Analysis

Let:

```
m = number of values stored for a specific key
n = total number of set calls
```

`set`

Appending to a list is `O(1)` amortized.

```
Time Complexity: O(1)
```

Each `set` stores one `(timestamp, value)` pair.

```
Space Complexity: O(1) per call
```

Overall storage after all calls:

```
Space Complexity: O(n)
```

`get`

For a key with `m` stored values, we binary search over its timestamp list.

```
Time Complexity: O(log m)
```

No extra data structures are created.

```
Space Complexity: O(1)
```

---

5. Alternative Approaches

---

1. Linear Search

For each `get`, scan all timestamps for the key and find the best one.

```
Time Complexity:
set: O(1)
get: O(m)
```

This is too slow when there are many calls.

---

2. Hash Map + Built-in `bisect`

Instead of writing binary search manually, we can use Python's `bisect` module.

Example idea:

```
bisect_right(timestamps, timestamp)
```

This finds the insertion position after all timestamps `<= timestamp`.

However, we would usually store timestamps and values separately:

```
key -> timestamps list
key -> values list
```

Then `get` can do:

```
idx = bisect_right(timestamps, timestamp) - 1
```

This is also efficient:

```
set: O(1)
get: O(log m)
```

---

3. Balanced Binary Search Tree

In languages like Java or C++, we could use:

```
key -> TreeMap(timestamp -> value)
```

Then use something like:

```
floorKey(timestamp)
```

This gives the greatest timestamp less than or equal to the query timestamp.

Complexity:

```
set: O(log m)
get: O(log m)
```

But in Python, the hash map plus list plus binary search approach is simpler and faster for this problem.